



Integrantes:

-Alfonso Luna

-Felipe Becerra

-Iván Del Pino

Ramo: Taller Aplicado de programación

Sección: 002_V

INDICE

Contenido

INDICE	2
1. INTRODUCCIÓN	3
2. OBJETIVO GENERAL:	4
3. OBJETIVOS ESPECÍFICOS:	4
4. DEFINICIÓN DE REQUERIMIENTOS	5
5. DESCRIPCIÓN TÉCNICA	7
6. METODOLOGÍA Y PLANIFICACIÓN	8
7. PLANIFICACION SEMESTRAL	9
8. GRÁFICOS	11
9. MOCKUPS/WIREFRAMES.....	15
10. FUNCIONALIDADES DE NEGOCIO	17
11. FUNCIONALIDADES BASE.....	18
12. MECANISMOS DE CONTROL Y CALIDAD.....	20
13. DEFINICIÓN DE LOS CASOS DE PRUEBAS	21
Backend	21
Frontend.....	28
14. Detalle de Base de datos de Prueba.....	43
15. Mejoras pertinentes	44
16. CONCLUSIÓN.....	47

1.INTRODUCCIÓN

El auge de la personalización de datos en el entorno digital ha transformado profundamente la forma en que los usuarios consumen y se relacionan con el contenido multimedia. Desde la masificación de funciones de recopilación anual y analítica en tiempo real en plataformas de música y lectura, se ha evidenciado una clara tendencia de consumo: las personas no solo buscan experimentar el arte, sino también registrar, comprender y compartir sus propios hábitos culturales dentro de una comunidad. Sin embargo, en el ámbito de la industria cinematográfica actual, existe un vacío operativo que impide capitalizar de forma integrada el verdadero potencial de la analítica personal aplicada a las películas.

Con el propósito de cubrir este nicho de mercado nace KinoStats, una plataforma digital de analítica cinematográfica personal orientada a optimizar la experiencia de registro, descubrimiento y organización de perfiles cinéfilos. El presente informe detalla el proceso de diseño y construcción del sistema, abarcando desde el levantamiento de los objetivos estratégicos y la planificación del proyecto mediante diagramas de Gantt, hasta la definición de historias de usuario y maquetas visuales (mockups). Asimismo, se expone en profundidad el desarrollo de su arquitectura de software, detallando tanto las funcionalidades base de autenticación y persistencia como los módulos de negocio principales: la bitácora personalizada, el motor de estadísticas avanzadas y la integración en tiempo real con catálogos internacionales de la industria.

2.OBJETIVO GENERAL:

Lanzar al mercado una plataforma digital de analítica cinematográfica personal que optimice la experiencia del usuario al registrar, descubrir y compartir sus hábitos cinéfilos, garantizando una alta disponibilidad del servicio (uptime del 95%) y ofreciendo un mínimo de 5 métricas de valor que fomenten el engagement de la comunidad.

3.OBJETIVOS ESPECÍFICOS:

Estructurar una arquitectura de datos e información centralizada que garantice la consistencia, el almacenamiento seguro y la gestión eficiente de perfiles cinematográficos y listas personalizadas de los usuarios.

Enriquecer el catálogo de la plataforma mediante la integración automatizada de metadatos cinematográficos globales, asegurando que el usuario disponga de información actualizada y detallada de la industria en tiempo real.

Implementar un modelo predictivo basado en inteligencia artificial que personalice las recomendaciones de contenido según los hábitos históricos de consumo, incrementando el tiempo de retención del usuario dentro de la app.

Diseñar una interfaz de usuario interactiva y centrada en la experiencia del cliente (UX), facilitando la visualización clara de estadísticas clave y simplificando el proceso de registro de películas.

Asegurar la escalabilidad operativa y la confianza del cliente mediante el despliegue en un entorno cloud de alta disponibilidad y la adopción de estándares robustos de seguridad para la protección de la identidad digital del usuario.

4.DEFINICIÓN DE REQUERIMIENTOS

H1- Autenticación de usuarios

“Como usuario quiero poder crear una cuenta para acceder de forma segura a mi historial personal de reseñas creadas, y poder recuperarla en caso de ser necesario. ”

Requisitos funcionales:

- RF-001 El sistema debe permitir el registro de nuevos usuarios con nombre de usuario, correo y contraseña válidos, validando también unicidad de username y email
- RF-002 El sistema debe permitir cambiar la contraseña del usuario, validando la contraseña actual
- RF-003: El sistema debe permitir el cierre de sesión, eliminando la información local de esta.
- RF-004 El sistema debe permitir solicitar la recuperación de contraseña vía email.
- RF-005 El sistema debe permitir restablecer la contraseña mediante un código válido y vigente.
- RF-017: El sistema debe permitir la autenticación de usuarios registrados y proteger los endpoints sensibles requiriendo autenticación

Requisitos no funcionales:

- RNF-001 La contraseña creada está hasheada con Bcrypt para no exponer información delicada sobre los perfiles de usuario.
- RNF-002 El sistema debe proteger los endpoints sensibles mediante autenticación.
- RNF-003: Las pantallas de autenticación y creación de cuenta deberán enmascarar en tiempo real los caracteres de las contraseñas ingresadas, protegiendo la confidencialidad del usuario frente a observadores externos.

Criterios de aceptación:

- Se debe poder ingresar a la app con un usuario y una contraseña
- La contraseña debe tener un mínimo de 8 caracteres entre letras, números y caracteres especiales
- El usuario debe poder actualizar su correo y contraseña.

H2 - Gestión de registros

“Como usuario, quiero registrar, clasificar y reseñar las películas que he visto, indicando la fecha exacta, para mantener un diario detallado y personal de mi actividad cinematográfica.”

Requisitos funcionales:

- RF-006 El sistema debe permitir registrar logs (reseñas) de películas con fecha obligatoria y un rating y reseña opcionales
- RF-007 El sistema debe permitir consultar los logs de un usuario, ordenados por fecha
- RF-008 El sistema debe permitir actualizar y eliminar los existentes

Requisitos no funcionales

- RNF-004: Cuando se guarde la reseña el sistema debe responder en un tiempo estimado de 5 segundos

Criterios de aceptación

- Para crear un registro, el usuario tendrá como campo obligatorio la fecha de visualización.
- Si el usuario no escribe una reseña ni una calificación, el registro debe guardarse igualmente solo con la fecha.
- Las acciones destructivas (eliminar un registro) deben solicitar una confirmación previa al usuario mediante un cuadro de diálogo para evitar borrados accidentales.

H3-Dashboard de estadísticas:

“Como usuario, quiero visualizar las estadísticas de consumo (minutos totales, cantidad de películas, géneros predominantes) de forma gráfica para entender mis hábitos y el tiempo que invierto en el cine”

Requisitos funcionales:

- RF-012 El sistema debe generar estadísticas de consumo por mes y/o año del usuario
- RF-013 El sistema debe presentar un dashboard con watchlist y películas vistas recientemente.
- RF-014 El sistema debe permitir buscar películas y obtener sus detalles desde TMDb
- RF-015 El sistema debe permitir al usuario invocar el menú nativo de compartición del dispositivo para compartir estadísticas hacia otras aplicaciones.
- RF-016 El sistema generará una vista de estadísticas tipo wrapped

Requisitos no funcionales:

- RNF-005: La pantalla y los gráficos se adaptaran al estilo visual de KinoStats

Criterios de aceptación:

- Al compartir la estadísticas la APP creará un informe con un formato específico
- Los componentes visuales del dashboard deben manejar de forma segura los valores nulos o incompletos provenientes del backend sin provocar caídas del sistema
- Al exportar el resumen visual (Wrapped), la imagen generada no debe incluir los botones de la interfaz de usuario

H4 -Creación de listas:

“Como usuario, quiero poder crear y gestionar listas de películas para organizar mis títulos favoritos o aquellos que tengo pendientes por ver.”

Requisitos funcionales:

- RF-009 El sistema debe permitir crear, consultar y eliminar listas personalizadas de películas
- RF-010 El sistema debe permitir agregar y quitar películas de las listas.
- RF-011 El sistema debe crear automáticamente una watchlist al registrar un usuario

Requisitos no funcionales:

- RNF-006: Si durante la creación de una lista el usuario experimenta fallas de red, la aplicación debe interceptar la excepción de manera segura

Criterios de aceptación:

- El sistema debe impedir al usuario crear una lista sin nombre
- El usuario puede agregar un nombre a una lista teniendo como máximo de 50 caracteres
- La aplicación debe permitir la creación de listas con nombres idénticos diferenciándolas internamente por un ID
- Dentro de una lista el usuario podrá agregar o quitar una película

5.DESCRIPCIÓN TÉCNICA

STACK TECNOLÓGICO:

- Lenguajes: Backend - Java / App - Kotlin
- Framework: Spring Boot con Spring Security, Bcrypt y Spring Data JPA.
- Toolkit: Jetpack Compose (App móvil)
- Base de datos: Postgresql
- Networking: Retrofit para el consumo APIs externas
- Servicios externos: TMDB API

REPOSITORIO Y VERSIONAMIENTO:

El desarrollo del código se realizará en branches particulares para las distintas partes de la aplicación que se estén desarrollando. Una vez terminada se hará un push al repositorio de github para luego hacer un merge al main. Se utilizará versionado semántico.

Los enlaces de Github son:

- Fronted: <https://github.com/Alf-Luna/App-Stats-Peliculas.git>
- Backend: <https://github.com/Alf-Luna/Backend-Stats-Peliculas.git>

Ramas:

- main: Código que se utiliza para producción
- develop: rama de integración donde se unen las funcionalidades una vez terminadas antes de pasar a producción.
- feature: ramas temporales para el desarrollo, una vez terminadas se incorporan a develop mediante un push.

6. METODOLOGÍA Y PLANIFICACIÓN

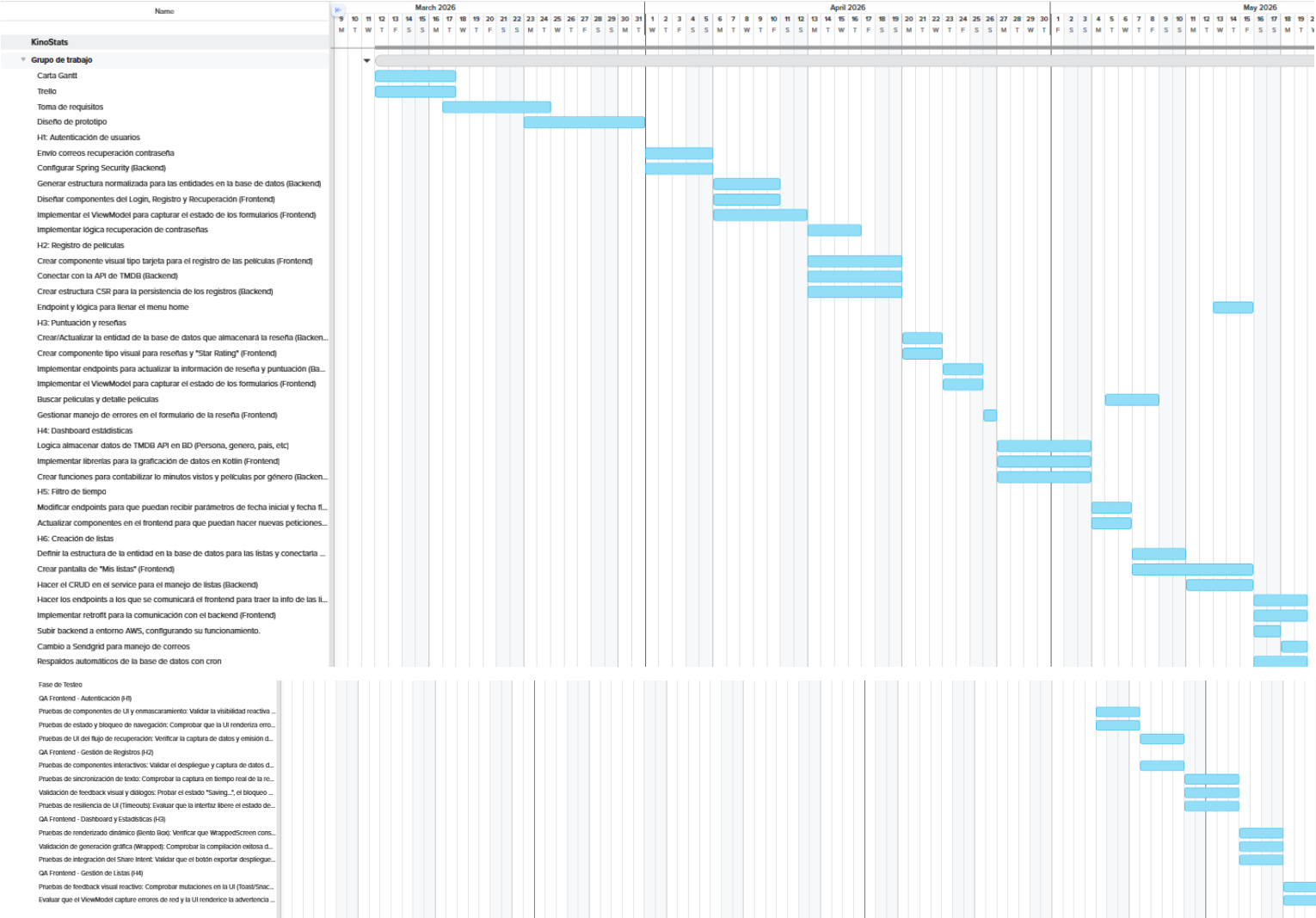
Ciclo de Vida del Software (Modelo Incremental)

Hemos seleccionado un modelo de desarrollo incremental para priorizar la entrega temprana de valor. Esta estrategia nos permite consolidar primero las funcionalidades base —como la gestión de usuarios y el registro del historial de películas—, construyendo los cimientos de datos necesarios antes de implementar el motor de estadísticas de KinoStats. Dada la ventana de tiempo del proyecto, este enfoque mitiga riesgos técnicos al facilitar la detección temprana de fallos de integración entre el *backend* y la aplicación móvil. Además, nos otorga una mayor flexibilidad arquitectónica para adaptar ambas capas ante posibles imprevistos.

Marco de Trabajo Ágil (Scrum)

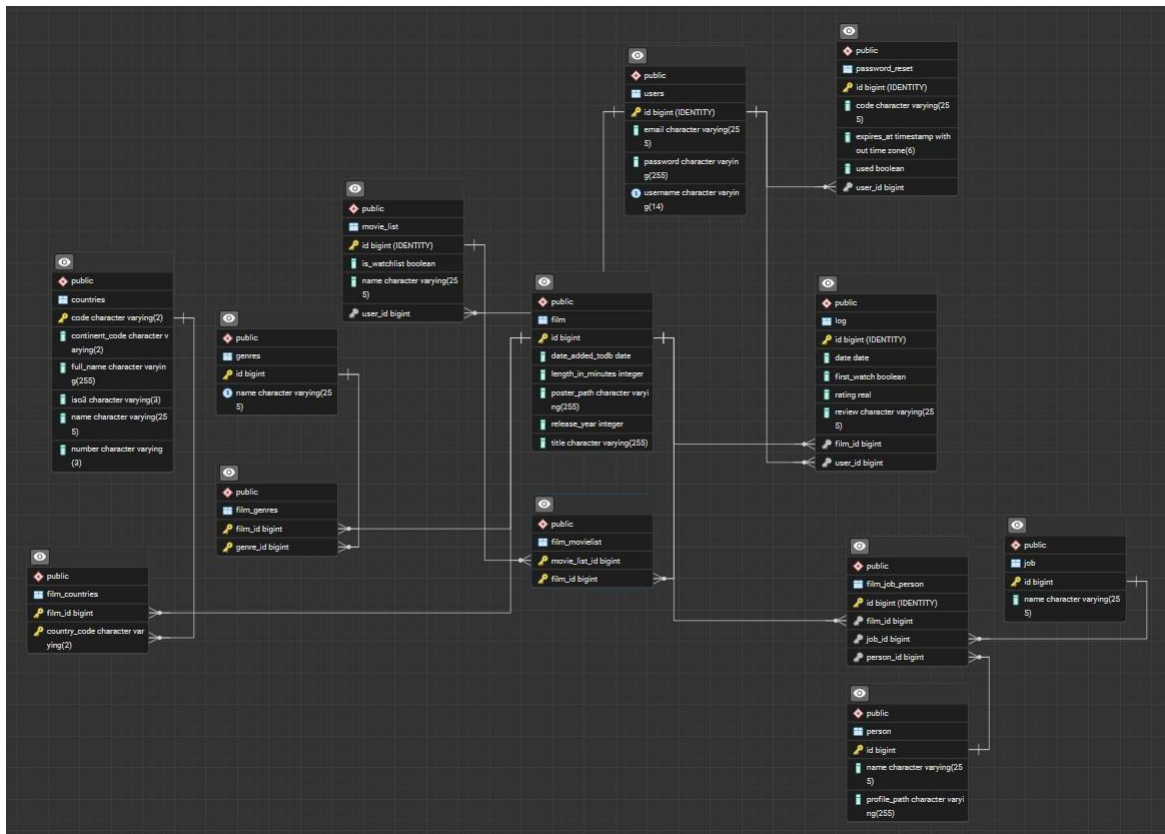
La ejecución del proyecto se regirá bajo el marco de trabajo Scrum. Estructuraremos el desarrollo en *Sprints* de dos semanas, iterando sobre historias de usuario y tareas priorizadas del *Product Backlog*. Para garantizar la alineación del equipo, realizaremos *Daily Scrums* de 15 minutos enfocados en sincronizar los avances diarios y levantar bloqueos. Al cierre de cada iteración, ejecutaremos la ceremonia de *Sprint Review* para validar el incremento de producto construido con los *stakeholders*, seguida de una *Sprint Retrospective* para

7.PLANIFICACION SEMESTRAL



8. GRÁFICOS

MODELO DE DATOS



MODELAMIENTO UML

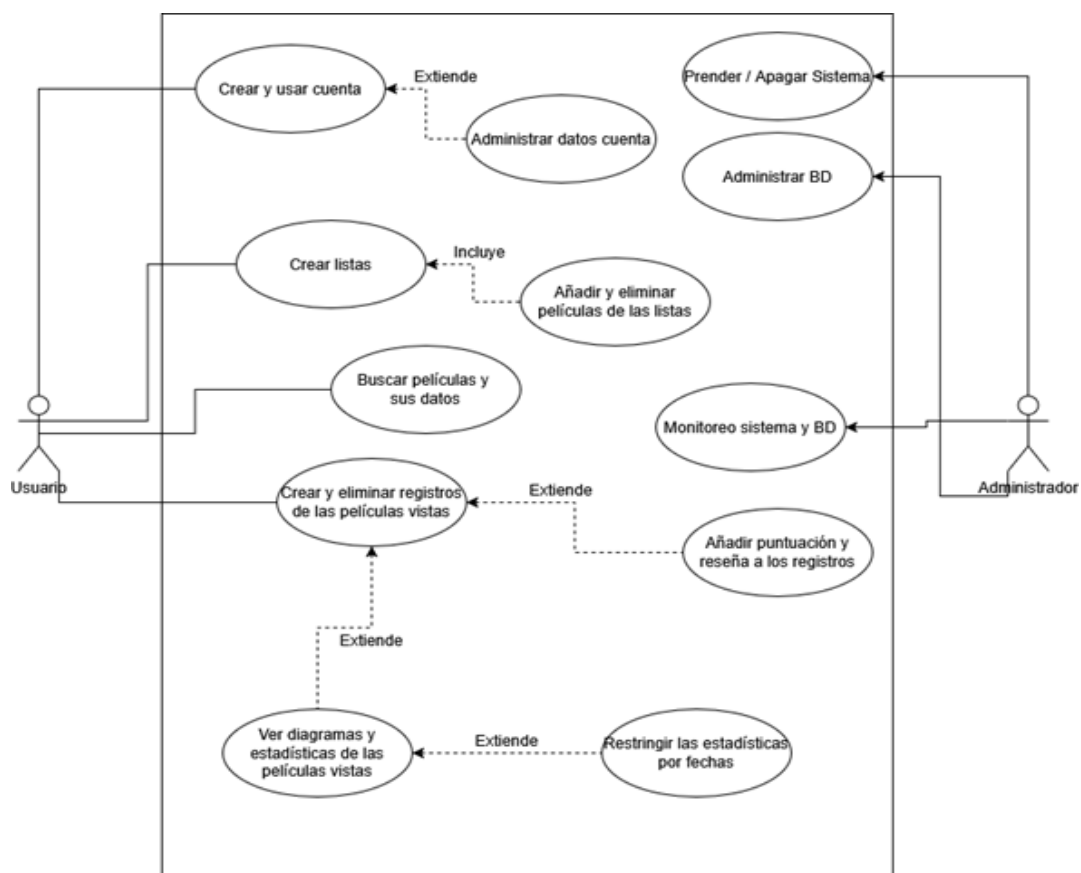
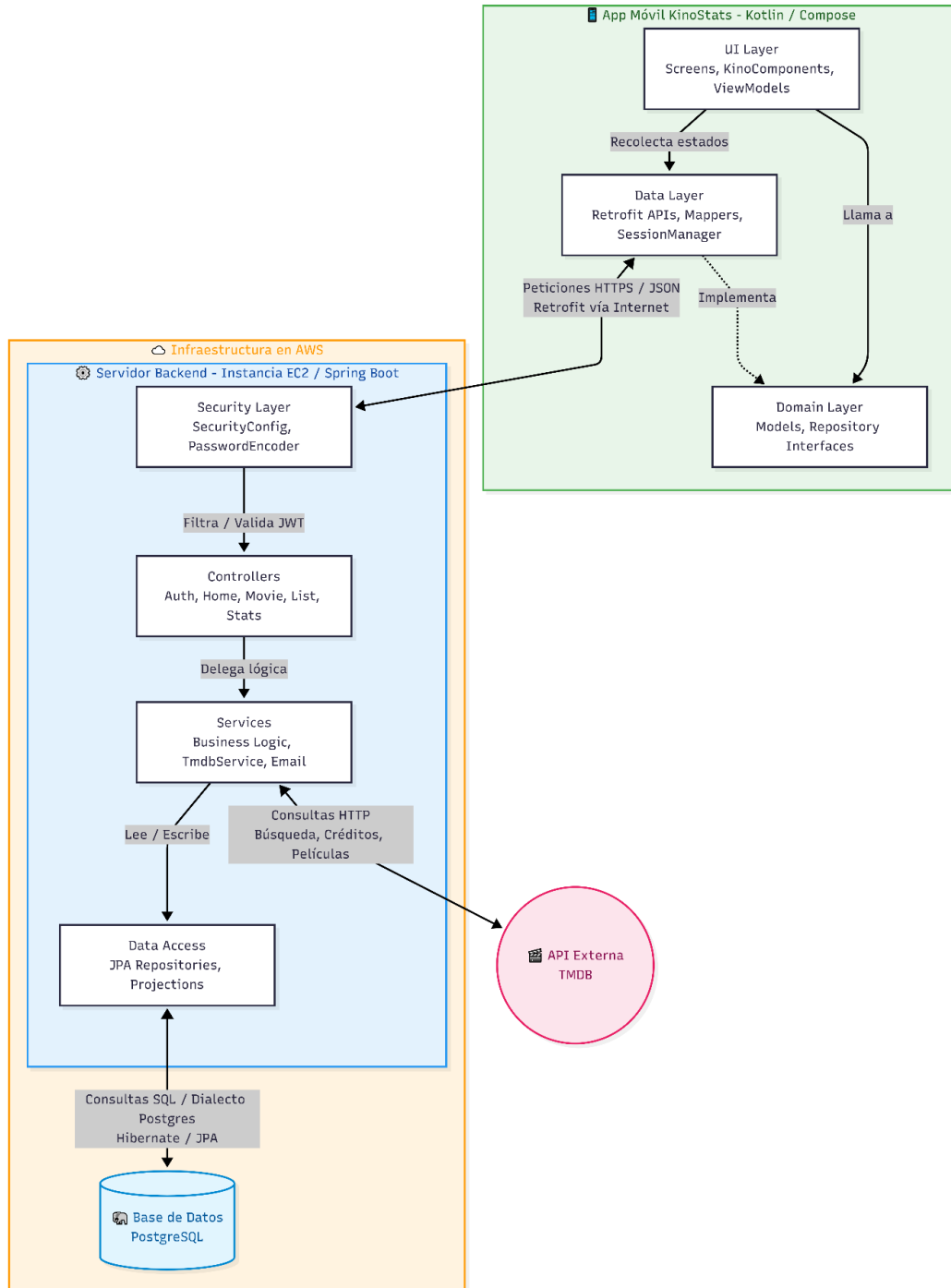


Diagrama de Arquitectura



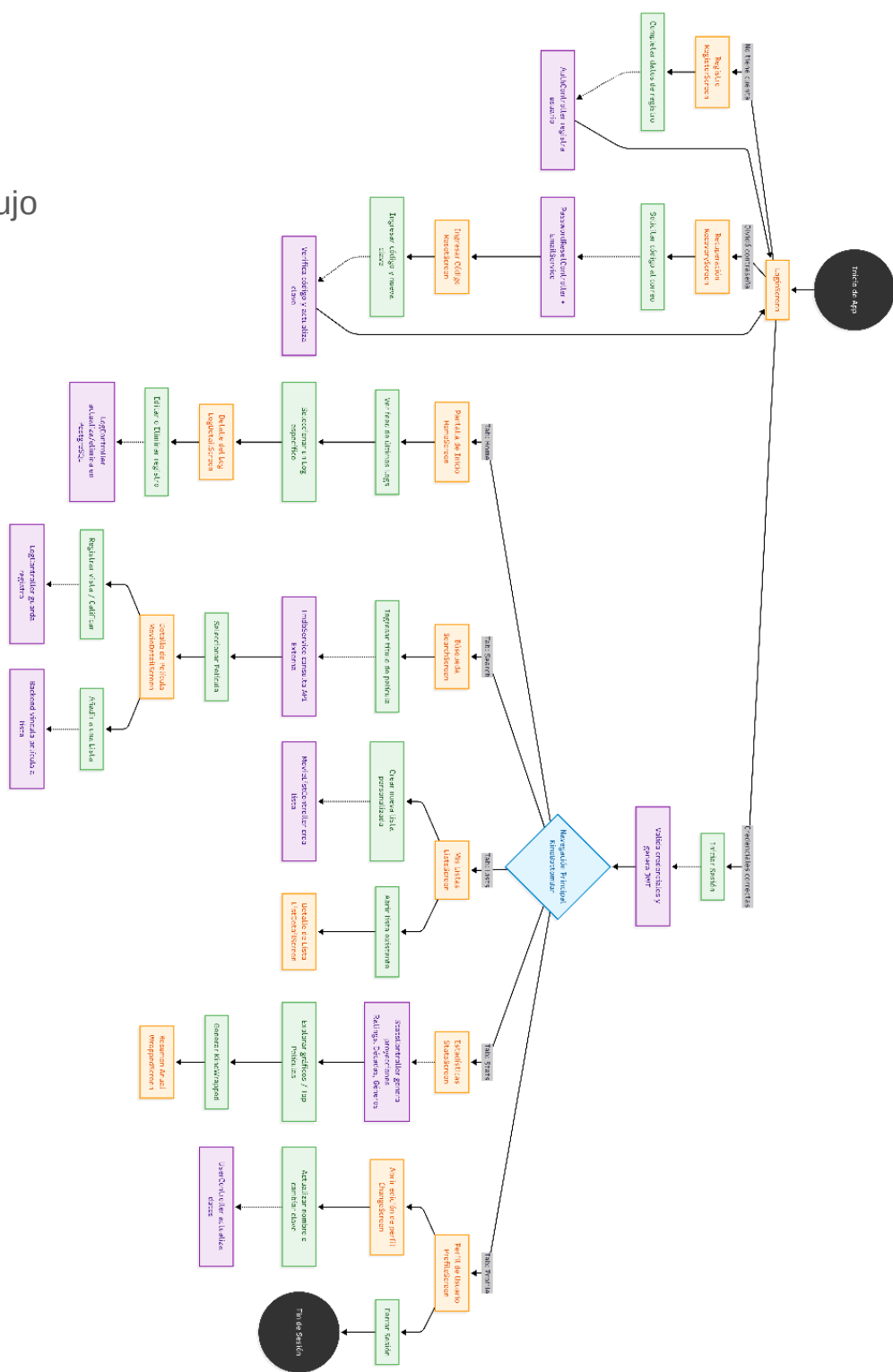
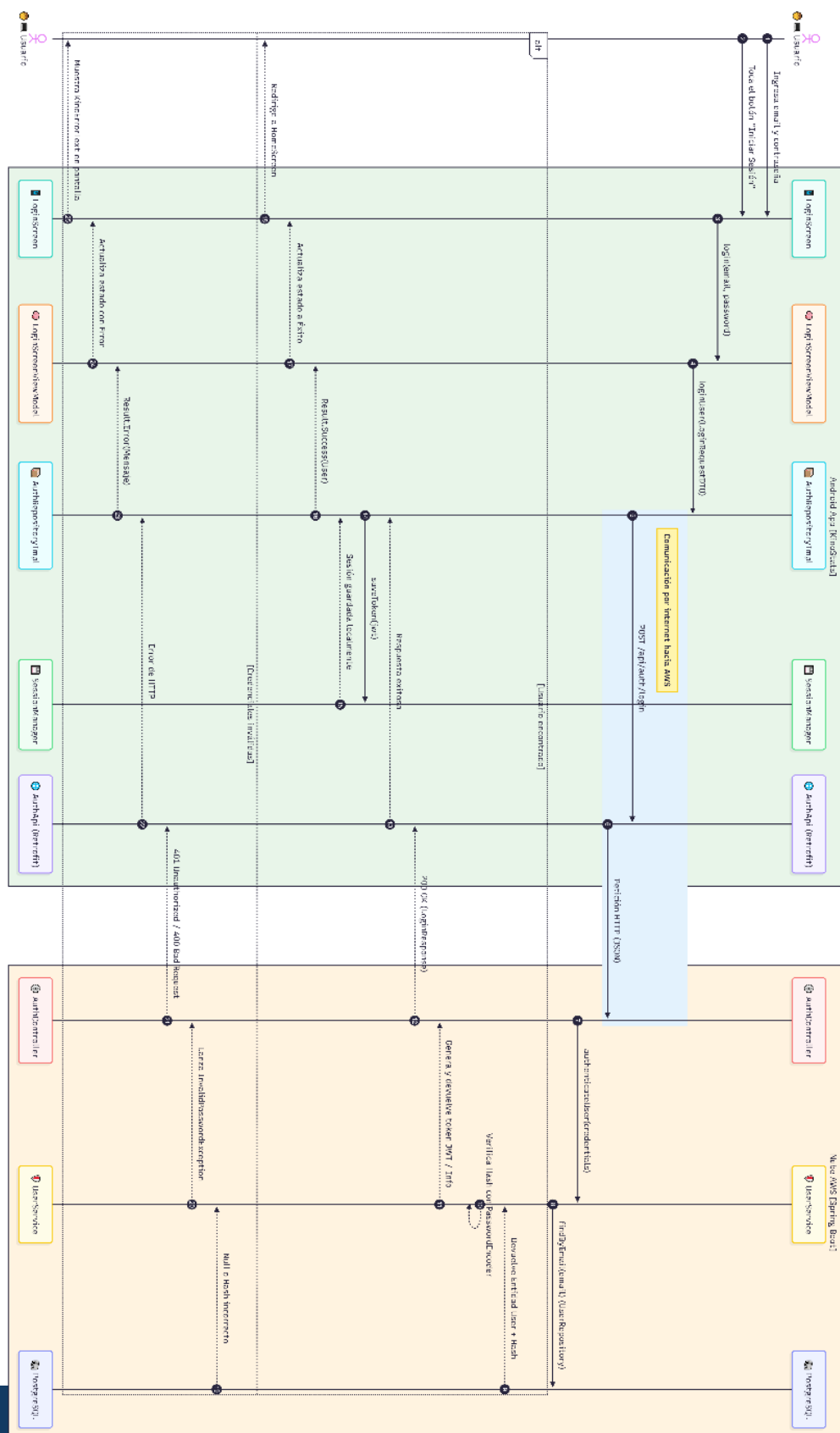
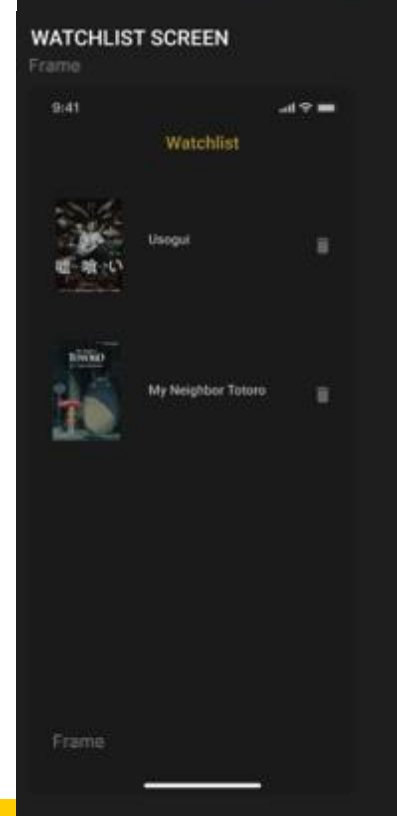
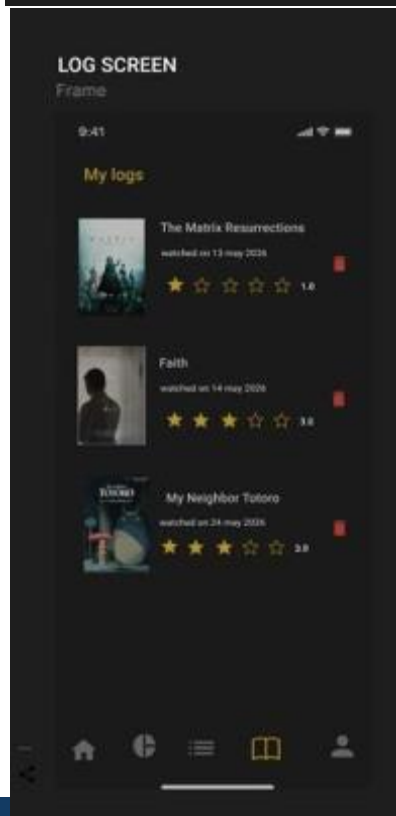
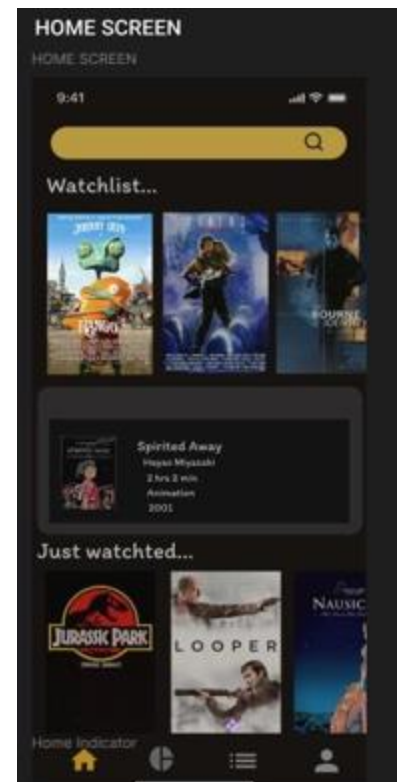
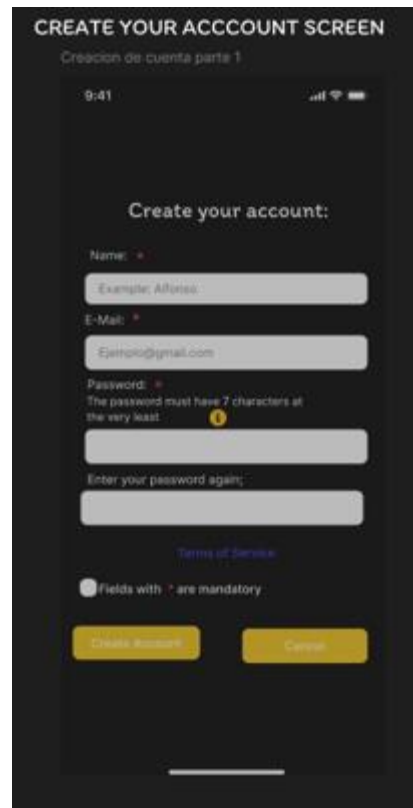
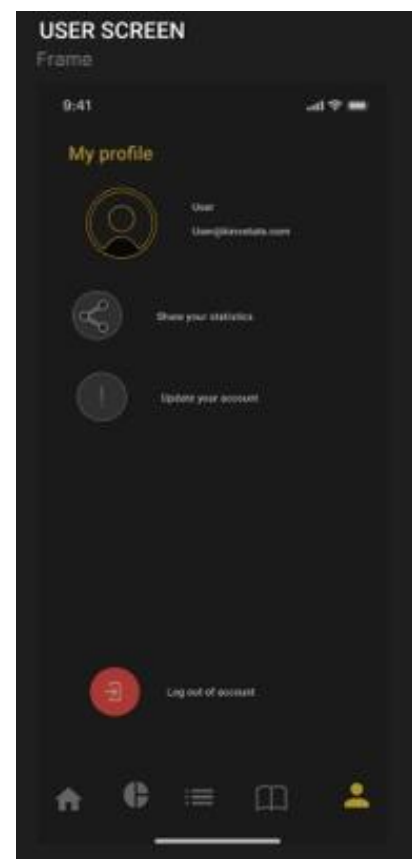
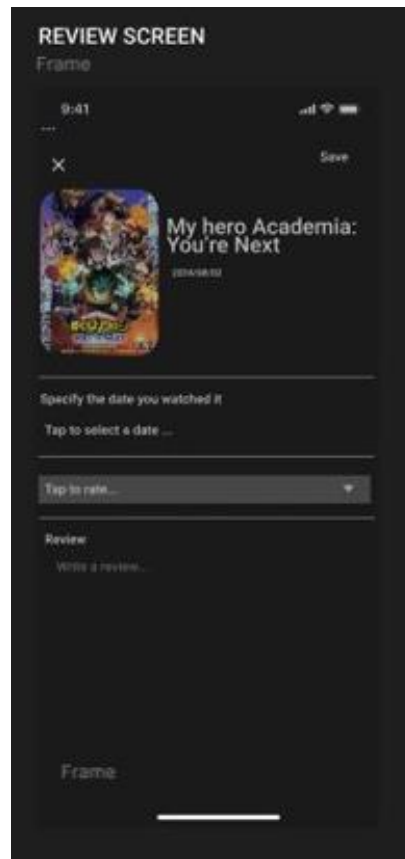
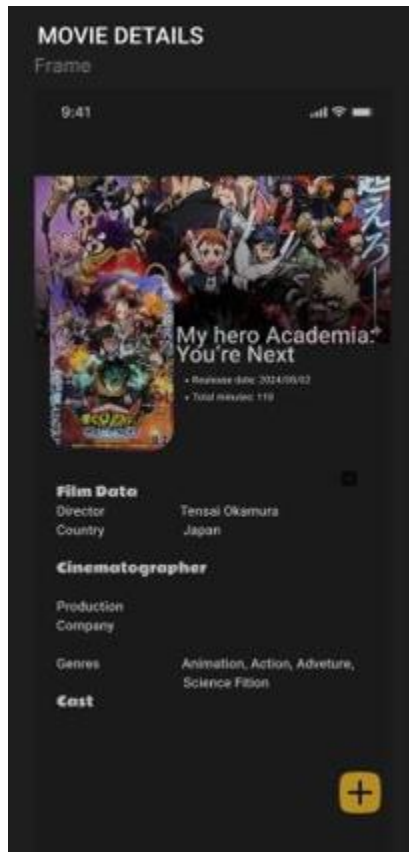


Diagrama de Flujo



9. MOCKUPS/WIREFRAMES





10. FUNCIONALIDADES DE NEGOCIO

1. Módulo Orquestador de Inicio (Dashboard / Home)

Esta es una adición crítica a la arquitectura. Se ha implementado el HomeController y HomeService para reducir la latencia y la cantidad de peticiones que el cliente móvil debe realizar al abrir la aplicación.

El endpoint GET /api/v1/home/{userId} actúa como un agregador de datos.

El servidor consulta internamente los repositorios de Bitácora, Listas y Estadísticas para ensamblar un HomeDTO unificado. Este objeto entrega de golpe la última película vista, la lista favorita del usuario y un resumen de las métricas del mes en curso.

2. Módulo de Registros o "Bitácora" (Logs)

Opera a través del LogController, ofreciendo operaciones CRUD completas. Recibe y procesa los visionados de películas, almacenando la fecha, la reseña, la calificación otorgada y registrando internamente metadatos vitales para futuras analíticas.

3. Módulo de Estadísticas (Stats)

Representa el motor analítico de KinoStats. A través del endpoint POST /api/v1/stats/get en el StatsController, el backend realiza la carga computacional pesada. Recibe parámetros de filtro temporal (mes/año) y devuelve un StatsResponseDTO con los datos consolidados: conteo de horas invertidas, distribución por décadas, orígenes geográficos y los rankings del top 5 de directores y actores más consumidos.

4. Módulo de Listas de Películas (Movie Lists)

El MovieListController provee la capacidad de organización personalizada. Soporta la creación de listas a la medida, la gestión intrínseca de la "Watchlist" (marcadora de películas pendientes) y facilita métodos atómicos para agregar o retirar obras individuales sin comprometer la integridad de la colección completa.

5. Módulo de Catálogo e Integración Externa (Movie Supplier)

El MovieSupplierController conecta a la plataforma con el ecosistema global del cine delegando las peticiones a la API de TMDB (The Movie Database).

Procesa búsquedas de texto mediante /search/{query}, devolviendo objetos ligeros para la carga rápida de portadas.

Extrae y formatea metadatos en profundidad (duración, directores, sinopsis completas) consultando directamente el ID de la obra mediante /api/v1/movies/{id}.

11. FUNCIONALIDADES BASE

El sistema de autenticación y gestión de identidades ha sido refactorizado para ofrecer mayor seguridad y acoplamiento exacto con las necesidades del cliente móvil.

1. Módulo de Registro de Usuarios

La creación de cuentas se mantiene centralizada en el UserController a través del endpoint POST /api/v1/users/add. Este controlador recibe los datos iniciales, los valida y delega al servicio correspondiente la encriptación de la contraseña y la inicialización del entorno del usuario (como la creación automática de su "Watchlist" predeterminada).

2. Autenticación y Sesión (Login)

La arquitectura ha evolucionado creando un controlador dedicado exclusivamente a la autenticación: el AuthController.

A través del endpoint POST /api/v1/auth/login, el backend recibe un objeto LoginRequest con las credenciales del usuario.

Tras validar la identidad, retorna un LoginResponse, estructurando formalmente la respuesta de acceso para el almacenamiento seguro de sesiones en el dispositivo móvil.

3. Sistema de Recuperación de Clave

Se ha implementado una solución completa y robusta para la recuperación de contraseñas olvidadas, orquestada por el PasswordResetController y el PasswordResetService.

Fase de Solicitud: El endpoint POST /api/v1/password-reset/request recibe el correo del usuario. Internamente, el sistema genera un código numérico seguro de 6 dígitos con validez temporal de 15 minutos. Este código es despachado automáticamente utilizando la nueva integración EmailService.

Fase de Restablecimiento: El endpoint POST /api/v1/password-reset/reset intercepta el código de verificación junto con la nueva contraseña, validando la transacción antes de aplicar los cambios en la base de datos.

4. Mantenedores de Usuario (Gestión de Perfil)

Para otorgar control granular al usuario sobre sus datos, el UserController ha fragmentado la lógica de actualización.

Modificación de información demográfica y preferencias a través de PUT

/api/v1/users/{id}/details, apoyado en el UserDetailsUpdateDTO.

Modificación de la credencial de acceso (requiriendo la clave actual) mediante PUT

/api/v1/users/{id}/password, apoyado en el UserPasswordUpdateDTO. Esta separación previene la sobrescritura accidental de datos sensibles.

12. MECANISMOS DE CONTROL Y CALIDAD

Certificación de Avances

Para asegurar que las funcionalidades construidas cumplen con los requisitos del docente/cliente, el equipo implementará los siguientes mecanismos de validación:

- **Seguimiento de la Carta Gantt:** Revisión periódica para contrastar los hitos planificados con el progreso real del desarrollo.
- **Demostraciones Funcionales:** Presentación de los avances del código en entornos de prueba para validar el comportamiento del software en tiempo real.
- **Plan de Pruebas Estructurado (QA):** Ejecución de ciclos de pruebas funcionales divididos en pruebas unitarias (para lógica de negocio aislada) y pruebas de interfaz/E2E (para validar flujos completos del usuario).
- **Entornos Aislados:** Uso de bases de datos de prueba locales para evitar la contaminación de datos y proteger la integridad de la información durante la fase de desarrollo.
- **Trazabilidad del Repositorio:** Revisión del historial de *commits* para evidenciar el trabajo continuo, la integración de características y el aporte individual al código fuente.

13. DEFINICIÓN DE LOS CASOS DE PRUEBAS

Backend

*Para más detalles en el plan de pruebas revisar archivo Plan Pruebas Backend

ID	Nombre	Descripción	Resultado Esperado	Estado Final
CP-01	Registro de Usuario Válido	Verificar que un usuario puede registrarse correctamente con datos válidos.	Respuesta 201 Created. Usuario persistido. Password almacenado con hash BCrypt (no en texto plano). Watchlist creada automáticamente para el nuevo usuario.	Exitoso
CP-02	Registro con username duplicado	Verificar que el sistema rechaza un registro con username ya existente.	Respuesta 400 Bad Request con body {"error": "Username already taken"}.	Exitoso
CP-03	Registro con email duplicado	Verificar que el sistema rechaza un registro con email ya existente.	Respuesta 400 Bad Request con body {"error": "Email already registered"}.	Exitoso

CP-04	Validación: username con menos de 3 caracteres	Verificar la validación de longitud mínima del username.	Prerrequisitos Aplicación desplegada.	Exitoso
CP-05	Validación: password con menos de 7 caracteres	Verificar la validación de longitud mínima de la contraseña.	Respuesta 400 Bad Request indicando error de validación en el campo password	Exitoso
CP-06	Validación: formato de email inválido	Verificar la validación del formato de email.	Respuesta 400 Bad Request indicando error de validación en el campo email.	Exitoso
CP-07	Login exitoso	Verificar que un usuario registrado puede autenticarse correctamente.	Respuesta 200 OK con LoginResponse que incluye userId, username, email y authorities [ROLE_USER].	Exitoso
CP-08	Login contraseña con incorrecta	Verificar que el sistema rechaza intentos de login con credenciales inválidas.	Respuesta 401 Unauthorized.	Exitoso
CP-09	Login credenciales sin	Verificar que el endpoint de login rechaza peticiones sin header de autenticación.	Respuesta 401 Unauthorized.	Exitoso
CP-10	Acceso a endpoint protegido sin autenticación	Verificar que los endpoints protegidos rechazan peticiones	Respuesta 401 Unauthorized.	Exitoso

		no autenticadas		
CP-11	Cambio de contraseña con password actual correcta	Verificar que un usuario puede cambiar su contraseña proporcionando la actual.	Respuesta 200 OK. Password actualizado en BD con nuevo hash BCrypt.	Exitoso
CP-12	Cambio de contraseña con password actual incorrecta	Verificar que se rechaza el cambio de contraseña si la actual no coincide.	Respuesta 401 Unauthorized con body {"error": "Incorrect password"}.	Exitoso
CP-13	Solicitar recuperación de contraseña con email válido	Verificar que se genera un código de recuperación al solicitar reset con email registrado.	Respuesta 200 OK. Registro password_reset creado con código de 6 dígitos, expiresAt a 15 minutos en el futuro y used=false. Llamada al servicio de email registrada.	Exitoso
CP-14	Solicitar recuperación con email no registrado	Verificar el comportamiento ante un email inexistente	Respuesta 404 Not Found.	Exitoso
CP-15	Restablecer contraseña con código válido	Verificar el restablecimiento de contraseña con un código vigente	Respuesta 200 OK. Password del usuario actualizado en BD. Código marcado como used=true.	Exitoso

CP-16	Restablecer contraseña con código inválido	Verificar que no se permite resetear con un código que no existe	Respuesta 400 Bad Request con body {"error":"Invalid code"}.	Exitoso
CP-17	Crear log de película válido	Usuario id=1 autenticado. Película id=550 en BD.	Respuesta 201 Created. Log persistido con todos los campos correctos.	Exitoso
CP-18	Crear log con rating superior al máximo (clamp)	Verificar que el rating se limita al máximo de 5.0 al crear un log.	Respuesta 201 Created. Rating persistido como 5.0.	Exitoso
CP-19	Crear log con rating inferior al mínimo (clamp)	Verificar que el rating se limita al mínimo de 0.5 al crear un log.	Respuesta 201 Created. Rating persistido como 0.5.	Exitoso
CP-20	Crear log con usuario inexistente	Verificar el manejo de error al referenciar un userId inexistente.	Respuesta 404 Not Found con body {"error":"User not found"}.	Exitoso
CP-21	Listar logs por usuario	Verificar la obtención de logs de un usuario ordenados por fecha descendente.	Respuesta 200 OK con lista de LogDTO ordenada por fecha descendente.	Exitoso
CP-22	Actualizar log existente	Verificar la actualización de un log existente	Respuesta 200 OK. Log actualizado en BD.	Exitoso
CP-23	Eliminar log existente	Verificar la eliminación de un log.	Respuesta 204 No Content. El log ya no existe en BD.	Exitoso

CP-24	Crear lista personalizada	Verificar la creación de una lista de películas personalizada.	Respuesta 201 Created con MovieListDTO. Lista persistida con isWatchlist=false.	Exitoso
CP-25	Watchlist auto-creada al registrar usuario	Verificar que al registrar un usuario nuevo se crea automáticamente su watchlist.	Entre las listas devueltas existe una con isWatchlist=true y nombre "Watchlist".	Exitoso
CP-26	Obtener listas de un usuario	Verificar la consulta de todas las listas de un usuario.	Respuesta 200 OK con lista de MovieListDTO.	Exitoso
CP-27	Agregar película a lista	Verificar la asociación de una película existente a una lista.	Respuesta 200 OK. La película aparece en la lista al consultarla.	Exitoso
CP-28	Quitar película de lista	Verificar la remoción de una película de una lista.	Respuesta 204 No Content. La película ya no aparece en la lista.	Exitoso
CP-29	Eliminar lista inexistente	Verificar el manejo de error al eliminar una lista que no existe.	Respuesta 404 Not Found.	Exitoso
CP-30	Generar estadísticas anuales	Verificar el cálculo de estadísticas para un año completo.	Respuesta 200 OK con StatsResponse DTO: conteo de películas, minutos vistos, listas por género/país/director/actor, ratings y décadas.	Exitoso

CP-31	Generar estadísticas mensuales	Verificar el cálculo de estadísticas para un mes específico.	Respuesta 200 OK con stats únicamente del mes de marzo de 2025.	Exitoso
CP-32	Estadísticas con usuario sin logs en el periodo	Verificar el comportamiento cuando el usuario no tiene actividad en el periodo.	Respuesta 200 OK con moviesWatched=0 y listas vacías.	Exitoso
CP-33	Home con usuario activo	Verificar la construcción del dashboard de home para un usuario con actividad.	Respuesta 200 OK con HomeDTO: watchlist (máximo 6), lastSeen no nulo, justWatched con últimas películas.	Exitoso
CP-34	Home con usuario sin actividad	Verificar el comportamiento del home para un usuario sin logs.	Respuesta 200 OK con lastSeen=null, justWatched=null y listas vacías	Exitoso
CP-35	Búsqueda de películas en TMDB (con resultados)	Verificar la búsqueda de películas con texto que retorna resultados.	Respuesta 200 OK con lista de TmdbMovieSearchResponse.	Exitoso
CP-36	Búsqueda sin resultados	Verificar el manejo de búsquedas que no encuentran películas.	Respuesta 200 OK con lista vacía.	Exitoso
CP-37	Obtener detalles de película	Verificar la obtención y mapeo correcto de detalles de película desde TMDB.	Respuesta 200 OK con MovieDetailsDTO que contiene título, director, actores (top 5), géneros y países concatenados con comas.	Exitoso

CP-38	Password almacenada con hash BCrypt	Verificar que la contraseña nunca se almacena en texto plano.	El valor en BD comienza con el prefijo \$2a\$ o \$2b\$ (BCrypt) y NO contiene la cadena mySecret123.	Exitoso
CP-39	Repository: findByEmail	Verificar el método personalizado findByEmail del UserRepository.	Devuelve un Optional<User> no vacío con los datos correctos.	Exitoso
CP-40	Repository: countMoviesWatched (query JPQL)	Verificar que la query JPQL personalizada compila y devuelve el conteo correcto.	Devuelve 2.	Exitoso
CP-41	Repository: findByIdAndIsWatchlistTrue	Verificar la búsqueda de la watchlist de un usuario.	Devuelve un Optional<MovieList> no vacío con isWatchlist=true y user.id=1.	Exitoso

Frontend

*Para más detalles en el plan de pruebas revisar archivo Plan Pruebas Frontend

ID	Nombre	Descripción	Resultado Esperado	Estado Final
CP-01	Validación de formato válido de correo electrónico	Evaluar que el componente encargado de la validación de formato (SRP) retorne un estado de éxito al recibir cadenas que cumplen con el	El sistema debe procesar la cadena mediante su expresión regular (Regex) y retornar un valor estrictamente verdadero (true), sin lanzar excepciones.	Exitoso

			estándar internacional de correos electrónicos (usuario@dominio.com)		
CP-02	Rechazo de formatos inválidos de correo electrónico	de de	Evaluar la robustez del validador de correos al inyectar datos malformados, verificando que rechace estructuras sin dominio, sin identificador o con caracteres no permitidos	El sistema debe identificar las anomalías sintácticas en todos los casos de prueba y retornar falso (false) iterativamente, previniendo el paso de datos corruptos hacia la base de datos.	Fallido
CP-03	Aprobación de contraseñas seguras	de	Verificar que la lógica de validación de contraseñas aprueba cadenas que cumplen estrictamente con los criterios de aceptación del negocio: mínimo de 7 caracteres, conteniendo letras, números y caracteres especiales.	El sistema debe retornar true para la cadena, permitiendo la creación de credenciales fuertes.	Exitoso
CP-04	Rechazo de contraseñas inseguras o incompletas	de o	Comprobar que el sistema rechaza de forma determinista las claves que comprometen la seguridad del usuario, ya sea por longitud insuficiente o por ausencia de la complejidad requerida (falta de números o caracteres especiales).	El validador debe analizar la longitud y la composición de los caracteres, retornando false de forma determinista al confirmar la ausencia de los criterios mínimos de seguridad, bloqueando la creación de la credencial.	Fallido

CP-05	Flujo de recuperación de contraseña	Comprobar el correcto funcionamiento de los eventos de la vista de recuperación, aislando la interfaz gráfica para garantizar la captura y emisión de datos.	La vista debe capturar la entrada de texto y actualizar el estado simulado. Al presionar el botón, se debe accionar el evento de recuperación.	Exitoso
CP-06	Purgado de datos locales al cierre de sesión	Validar que el caso de uso encargado del cierre de sesión interactúa correctamente con el almacenamiento local, asegurando la eliminación absoluta de los tokens de acceso para prevenir filtraciones de seguridad en sesiones compartidas.	Tras la orden de purgado, cualquier consulta posterior a la sesión debe retornar estrictamente un valor nulo (null), garantizando que la memoria local quedó limpia.	Exitoso
CP-07	Bloqueo de navegación y despliegue de errores por formulario inválido	Validar que el RegisterScreen interactúe correctamente con el ViewModel. Al ingresar datos inválidos, el estado interno debe actualizarse para renderizar los mensajes de error visuales y bloquear la ejecución de la función de enrutamiento (onNavigateToHome).	La UI debe emitir el evento al ViewModel, el cual detecta la falla de validación y actualiza el estado. La interfaz debe mostrar los errores ("Please check the required fields") y el indicador de navegación debe mantenerse estrictamente falso	Exitoso

CP-08	Mapeo y encapsulamiento íntegro de credenciales en el repositorio	Comprobar que el AuthRepositoryImpl construye correctamente el Data Transfer Object (UserDTO), recibiendo la contraseña en texto plano y delegándola intacta a la capa de red simulada, respetando la directriz de delegar el hasheo (Bcrypt) al servidor backend.	El repositorio debe empaquetar la contraseña exactamente como fue ingresada, sin aplicar manipulaciones ni alteraciones criptográficas locales, garantizando la integridad del objeto de transferencia.	Exitoso
CP-09	Alternancia reactiva de enmascaramiento o visual de contraseñas	Verificar que el componente KinoTextField gestiona correctamente su estado interno de visibilidad (passwordVisible), aplicando por defecto la transformación visual de ocultamiento y reaccionando dinámicamente a las interacciones del usuario sobre el icono de control	El estado interno del componente debe alternar al hacer clic, disparando una recomposición (recomposition) automática que actualice el icono visual y retire la transformación gráfica (PasswordVisualTransformation) de los caracteres.	Exitoso

CP-10	Aceptación de puntajes incrementales fraccionados exactos	Validar que la función o clase encargada de procesar las calificaciones (RatingValidator - SRP) acepte y procese estrictamente valores numéricos de punto flotante que representen saltos exactos de 0.5 estrellas.	El sistema matemático debe comprobar el módulo de la fracción y retornar true para todos los valores del conjunto de prueba, autorizando su paso hacia la capa de presentación o persistencia.	Exitoso (Tras refactorización)
CP-11	Rechazo de puntajes fuera del límite permitido	Comprobar la solidez del validador de calificaciones (SRP) al inyectarle deliberadamente valores anómalos o "casos límite" (Edge Cases) que violen las restricciones métricas del negocio (Rango: 0.0 a 5.0, saltos de 0.5	El algoritmo debe retornar false o lanzar una excepción controlada en los 3 escenarios descritos, previniendo la corrupción de estado en los componentes visuales de estrellas de Jetpack Compose.	Exitoso
CP-12	Asignación de Puntuación Incremental (0.5 - 5.0)	Verificación de la integración del componente RatingDropdownSelector. Confirma que el usuario puede abrir el menú, visualizar opciones incrementales de a media estrella y seleccionar un valor	El componente RatingDropdownSelector debe desplegar correctamente el listado de opciones en intervalos exactos de media estrella. Al confirmar la selección del valor "4.5", el menú	Exitoso

		que impacte directamente en el estado y la interfaz visual.	debe cerrarse de manera automática y el estado de la aplicación debe actualizarse de forma determinista capturando el valor flotante exacto (4.5f). Como consecuencia de este cambio de estado, la interfaz gráfica debe realizar una recomposición inmediata, reemplazando el texto de inicialización ("Tap to rate...") por la puntuación numérica seleccionada y su respectiva representación visual en el componente de estrellas estáticas (StaticStars).	
CP-13	Transformación de Timestamp a formato regional de fecha	Verificar que la utilidad formateadora de fechas (SRP) convierte correctamente un Timestamp UTC en milisegundos a una cadena de texto amigable ("dd MMM yyyy"), indispensable para la lectura humana en la UI.	El sistema matemático debe realizar la conversión de zona horaria y retornar el texto formateado con exactitud, sin sufrir desplazamientos de días.	Exitoso (Tras refactorización)
CP-14	Selección y de Formateo	Verificación de la integración del componente de calendario (KinoCalendar).	El componente de calendario debe desplegarse visualmente en la interfaz tras la interacción inicial. Al confirmar la	Exitoso

	Fecha de Visualización	Comprueba que la UI responda a la interacción del usuario, capture el timestamp seleccionado, aplique la conversión a zona horaria UTC y formatee la fecha en el estado de la vista (LogDetailScreen State) sin descartar la información.	selección mediante la acción "Ok", el sistema debe capturar el timestamp subyacente, procesarlo hacia un objeto LocalDate con anclaje a la zona horaria UTC y aplicar la máscara de formato regional ("dd MMM yyyy"). Como respuesta final, el estado interno debe ocultar el modal del calendario y la interfaz gráfica debe reflejar la actualización, reemplazando el texto de inicialización por la cadena de fecha formateada.	
CP-15	Creación exitosa de un registro mínimo (Solo fecha)	Confirmar que el sistema permite la creación de un objeto de dominio Log válido aportando únicamente el campo obligatorio (fecha de visualización), tolerando explícitamente valores nulos en la reseña de texto y la calificación numérica.	El sistema debe construir el objeto de dominio correctamente, inicializando sus propiedades nulas sin arrojar excepciones de tipo NullPointerException ni fallos de validación.	Exitoso
CP-16	Sobrescritura de datos en la edición de un registro existente	Confirmar que la función de actualización (updateLog) reemplaza íntegramente los campos modificados por el usuario (texto de la reseña y puntuación)	La cantidad total de registros en el repositorio debe mantenerse estática (evitando duplicaciones) y el registro objetivo debe reflejar exactamente el nuevo texto suministrado	Exitoso

		manteniendo el identificador único intacto, sin generar registros duplicados en la base de datos.		
CP-17	Captura y persistencia temporal de texto en pantalla de detalle	Verificar la capacidad del componente LogDetailContent para recibir, procesar y retener la escritura del usuario en tiempo real, garantizando la comunicación bidireccional con el estado del ViewModel.	La interfaz debe procesar la entrada de texto sin latencia o descartes, disparando eventos que actualicen el StateFlow y redibujen el componente sin perder información	Exitoso
CP-18	Despliegue de feedback visual e interbloqueo de peticiones duplicadas	Comprobar que el sistema notifica visualmente al usuario al iniciar una petición de guardado a la base de datos, interrumpiendo transitoriamente la interactividad del botón principal para evitar inserciones múltiples por clics accidentales.	El sistema debe mutar el estado a carga (isSubmitting = true), transformando visualmente el botón a "Saving..." o desactivándolo, imposibilitando lanzar una segunda petición concurrente.	Exitoso
CP-19	Acceso Constante a Nuevo Registro de Película	Validación de la disponibilidad permanente de la acción de registro. Comprueba que el sistema permite desencadenar la creación de una nueva reseña en cualquier momento desde los detalles de la película, sin importar los registros previos,	El menú debe desplegarse y la acción "Add Log" debe desencadenar un evento de enrutamiento (onNavigateToLog) transmitiendo exclusivamente el identificador de la película (movieId), preparando el terreno para una instancia	Exitoso

		cumpliendo con la capacidad de múltiples entradas.	completamente nueva de registro.	
CP-20	Prevención de borrado accidental mediante diálogo de confirmación	Comprobar que el intento de eliminación detiene el flujo lógico y renderiza una barrera de confirmación visual (KinoDeleteDialog), ejecutando la petición destructiva (borrado en BD) únicamente si el usuario confirma de forma explícita.	El primer clic debe mutar el estado visual para abrir la advertencia, protegiendo los datos. Solo el segundo clic (la confirmación explícita) debe desencadenar la petición de eliminación al repositorio simulado.	Exitoso
CP-21	Aborto automático de solicitud de red por Timeout (5s)	Evaluar la capacidad del componente de persistencia y la UI para cancelar corrutinas estancadas tras superar el límite establecido de 5000 milisegundos, liberando la interfaz del bloqueo infinito de estado "Saving...".	El sistema de corrutinas debe lanzar un TimeoutCancellationException interno, el cual será atrapado por el ViewModel para retirar el bloqueo visual, regresar el botón a la normalidad y advertir al usuario del fallo de conexión.	Exitoso (Tras corrección de Bug)
CP-22	Formateo de minutos totales a escala métrica legible (Xh Ym)	Validar la función matemática que convierte la sumatoria bruta de tiempo de visualización (ej. 135 minutos) a una cadena de texto fraccionada y legible, indispensable para la caja de métricas del Dashboard.	El sistema debe calcular el cociente y el resto correctamente, retornando "2h 15m" para el primer valor y "0h 45m" para el segundo, sin errores de redondeo.	Exitoso
CP-23	Cálculo de extremos temporales	Verificar que la función encargada de procesar las estadísticas iterando sobre la colección de	El algoritmo debe ordenar o escanear la lista correctamente,	Exitoso (Refactorizado)

	(Primera y Última película)	registros identifica correctamente el objeto con el Timestamp más antiguo (Primera) y el más reciente (Última), incluso si la lista llega desordenada.	retornando los nombres/IDs exactos que corresponden a los Timestamps extremos, ignorando los registros intermedios.	
CP-24	Asignación de fallback ante géneros nulos (Tolerancia a fallos)	Comprobar que la capa de mapeo inyecta un valor de respaldo seguro ("Desconocido" o "Unknown") cuando la respuesta del backend omite la taxonomía de género de una película, previniendo caídas (crashes) en el gráfico de torta/barras.	El sistema no debe lanzar una NullPointerException. En su lugar, debe asegurar la cadena y asignarle el valor por defecto configurado para que los componentes gráficos puedan pintarse sin incidentes	Exitoso
CP-25	Reagrupación de registros mediante filtro temporal (Este Mes)	Comprobar que el StatsScreenView Model procesa correctamente los eventos de la interfaz y muta el estado de presentación (UI State), asignando de forma segura el año y mes seleccionados para la futura consulta al servidor, garantizando que	La colección devuelta debe contener únicamente los registros correspondientes al mes de junio de 2026, descartando los demás sin eliminarlos de la base de datos principal.	Exitoso

		la UI refleje el filtro activo.		
CP-26	Reagrupación de registros mediante filtro temporal (Este Año)	Comprobar que el StatsScreenView Model procesa correctamente la intención de ampliar la búsqueda a "Este Año", mutando el estado para retener el año actual y asignando un valor nulo al parámetro del mes, instruyendo así al backend para que no restrinja los datos a un mes específico.	La colección debe incluir tanto los registros de junio como los de meses anteriores del mismo año (ej. febrero 2026), descartando exclusivamente los registros de años pasados (ej. 2025).	Exitoso
CP-27	Renderizado dinámico de métricas y gráficos (Bento Box)	Verificar que la pantalla WrappedScreen consume correctamente los flujos de datos (StateFlow) provenientes del ViewModel y dibuja los componentes visuales interactivos (gráficos y contadores) adaptándose a la información provista.	La interfaz no debe colapsar ante la estructura de la Bento Box y debe reflejar fielmente los datos inyectados, cumpliendo con la directriz visual (KinoStats style)	Exitoso
CP-28	Bloqueo estricto de creación de	Asegurar que el sistema (ViewModel)	El sistema debe utilizar funciones como isBlank() para detectar la anomalía	

	lista con nombre vacío	impide la persistencia de una lista personalizada si el usuario no provee un nombre o solo introduce espacios en blanco.	y retornar un error o detener el flujo inmediatamente, impidiendo la llamada al repositorio.	
CP-30	Creación exitosa de listas concurrentes con nomenclatura idéntica	Verificar que el sistema permite la instanciación y persistencia de múltiples listas con el mismo nombre ingresado por el usuario, asignándoles identificadores únicos internos (UUID o autoincrementales) para evitar colisiones en la base de datos.	El sistema debe autorizar la transacción sin lanzar errores. La colección resultante debe contener dos listas llamadas "Mis Favoritas", pero cada una debe poseer un identificador de memoria o base de datos estrictamente diferente.	Exitoso
CP-31	Gestión del estado de UI al agregar una película a la lista	Comprobar que el MovieDetailView Model delega correctamente la acción de añadir una película al repositorio y muta el estado de la interfaz (UI State) de forma reactiva para mostrar un mensaje de éxito cuando la	El estado del ViewModel debe actualizarse asignando el mensaje de éxito correspondiente ("Added to [Nombre de la lista]"), sin realizar cálculos estructurales de	Exitoso

		operación en el servidor es confirmada.	matrices de forma local.	
CP-32	Disminución estructural y eliminación de una película	Validar el flujo de gestión de listas garantizando que, al eliminar una película, el sistema solicita una actualización de estado al backend, reflejando correctamente la reducción de elementos en la interfaz sin retención de datos.	Tras la ejecución, el estado del ViewModel debe transicionar exitosamente, reflejando una lista sin elementos (movieList.movies.isEmpty()), confirmando la sincronización post-eliminación.	Exitoso
CP-33	Manejo seguro de errores de red en la creación de listas	Evaluar que la arquitectura del sistema intercepta y gestiona de manera segura las excepciones de red (IOException o similares) durante la ejecución de tareas asíncronas en el ListRepository, evitando la propagación de errores no controlados que comprometan la estabilidad del	El error de red debe ser interceptado en la capa de datos. El sistema debe responder de manera controlada (retornando un estado de fallo sin lanzar excepciones al hilo principal), garantizando la continuidad operativa de la aplicación.	Exitoso

		proceso o la integridad de la aplicación.		
CP-34	Actualización visual y feedback al guardar película en lista	Comprobar que al interactuar con el botón "Save to List" desde los detalles de una película, la UI emite el evento correctamente, el estado procesa la inserción y se despacha un feedback visual inmediato (como un Toast o Snackbar) informando del éxito.	El sistema debe resolver la acción en menos de un segundo y actualizar la UI de manera reactiva para confirmar que el título ya forma parte del listado personal del usuario.	Exitoso
CP-35	Compilación de Bitmap para resumen visual (Wrapped)	Validar que la arquitectura del sistema inicializa y aísla correctamente el estado de los datos del Wrapped (WrappedScreen State). Al separar la información estadística de la lógica interactiva de la UI, se garantiza que el motor gráfico pueda consumir un estado limpio para la posterior compilación dinámica de la imagen.	El sistema debe instanciar un estado neutral (sin carga ni errores) listo para recibir la inyección de datos estadísticos, demostrando la separación de responsabilidades (datos vs. UI).	Exitoso

CP-36	Invocación de menú nativo de compartición (Share Intent)	Validar que al pulsar el botón de compartir, la app delega la imagen al sistema operativo.	El componente debe delegar la acción al sistema operativo abriendo la bandeja nativa de redes sociales y mensajería de Android.	Exitoso
CP-37	Creación y coexistencia de múltiples listas personalizadas	Comprobar que el sistema soporta la instanciación de más de una lista de películas por usuario, verificando que el repositorio de datos permita almacenar y recuperar múltiples entidades independientes simultáneamente.	El repositorio debe procesar ambas inserciones de manera consecutiva y la consulta final debe retornar una colección con un tamaño (size) exactamente igual a 2.	Exitoso
CP-38	Flujo maestro E2E: Registro, Búsqueda, Gestión de Listas y Reseñas	Prueba de Integración Completa (<i>End-to-End</i>). Validar la estabilidad del viaje del usuario desde la creación de una cuenta nueva, la búsqueda de una película en TMDB, su guardado en una lista personalizada (incluyendo la creación de dicha lista en tiempo real), hasta la generación de un registro (Log) con fecha dinámica, culminando con la verificación de la persistencia visual de los datos en las pantallas principales.	Resultado esperado: El framework de automatización (Compose UI Test / Espresso) debe ser capaz de completar el flujo íntegro sin sufrir colisiones (<i>crashes</i>), bloqueos de interfaz por teclados superpuestos, ni excepciones por <i>Timeout</i> . Las acciones asíncronas deben resolverse correctamente y la interfaz gráfica debe reflejar los componentes visuales de la película y la lista recién creada.	Exitoso

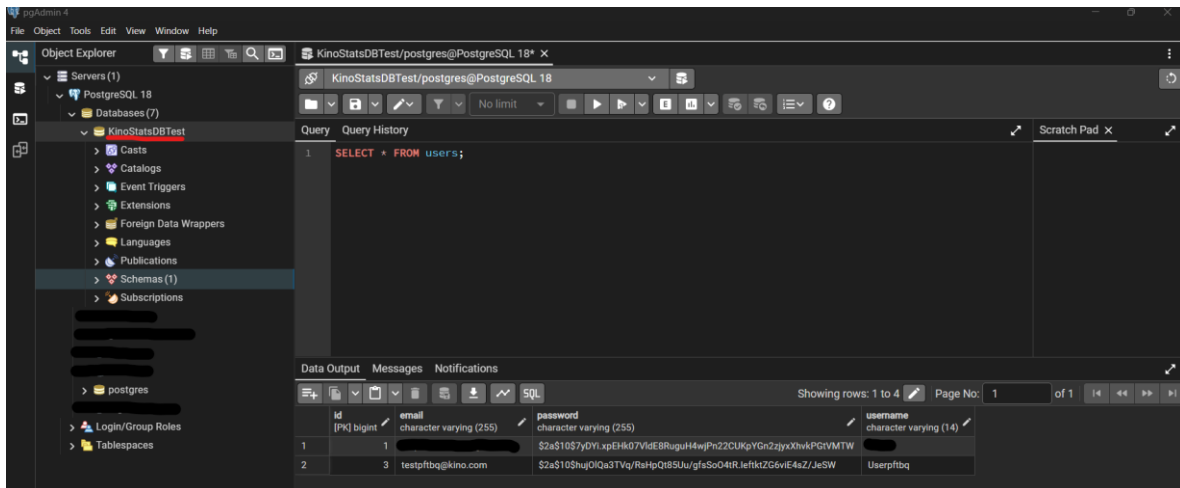
14. Detalle de Base de datos de Prueba

Para garantizar la máxima integridad de la información y evitar cualquier riesgo de contaminación o alteración de datos de producción, la totalidad de la suite de pruebas de validación (que lo requieran) fue ejecutada sobre un entorno de red estrictamente local.

La arquitectura de este entorno de pruebas se compone de los siguientes elementos:

- **Servidor Backend Local (Localhost):** La API de KinoStats se desplegó y ejecutó en un servidor local durante los ciclos de prueba. Esto permitió que la aplicación Android (Cliente) realizara las peticiones HTTP y simulara transacciones reales a través de la red local (Wi-Fi), garantizando un entorno controlado.
- **Base de Datos Aislada (PostgreSQL):** El servidor local fue conectado a una instancia de base de datos PostgreSQL independiente y dedicada exclusivamente a tareas de aseguramiento de calidad (QA). Esta base de datos funcionó aislando por completo la información de prueba de los esquemas de producción.
- **Estrategia de Datos Dinámicos:** Al contar con un entorno de base de datos local y efímero, los *scripts* de pruebas automatizadas (como el registro de cuentas o la creación de listas de películas) pudieron generar e inyectar registros dinámicos de forma segura (ej. nombres de usuario con sufijos aleatorios). Esto permitió validar el ciclo completo de persistencia de datos.
- Para las **pruebas de backend** se utiliza H2 Database en modo embebido (en memoria) operando en compatibilidad con PostgreSQL, motor del entorno productivo de KinoStats. El esquema se genera automáticamente desde las entidades JPA al iniciar el contexto, y los datos semilla se cargan mediante el script `data.sql`. Cada prueba se ejecuta dentro de una transacción con rollback automático, garantizando aislamiento entre tests. La base contiene los datos maestros del sistema (2 jobs, 19 géneros y un subconjunto de países), dos usuarios de prueba con contraseña hasheada en BCrypt, dos películas pre-cacheadas (*Spirited Away* y *Fight Club*) que simulan registros previamente obtenidos desde TMDb, una watchlist asociada al usuario principal y dos logs distribuidos en 2025 para validar las queries de estadísticas. Esta configuración permite ejecutar la suite completa en pocos segundos sin requerir un servidor de base de datos externo, manteniendo reproducibilidad total entre máquinas y entornos de integración continua.

Imagen de la BD local desde PgAdmin4



15. Mejoras pertinentes

A partir de los resultados obtenidos durante la ejecución de los ciclos de pruebas funcionales (Unitarias, de Integración y automatización *End-to-End*), se identificaron e implementaron diversas mejoras críticas. Estas modificaciones buscaron dar cumplimiento estricto a los estándares de calidad de software, seguridad de la información y experiencia de usuario de KinoStats:

- 1. Refactorización Arquitectónica y Calidad de Código (SRP):** Durante las pruebas unitarias, se detectó que los componentes de la capa de presentación (*ViewModels*) estaban sobrecargados con responsabilidades matemáticas y de transformación de datos. Para dar cumplimiento al Principio de Responsabilidad Única (SRP), se extrajo la lógica de cálculo de puntuaciones (*isRatingValid*) y el formato regional de fechas hacia validadores de dominio puros. Esto desacopló la interfaz gráfica de las reglas de negocio, mejorando drásticamente la mantenibilidad y testeabilidad del sistema

```

3 Usages
fun isRatingValid(rating: Float): Boolean {
    if (rating < 0.5f || rating > 5.0f) return false
    return rating.rem( other = 0.5f) == 0f
}

```

2 Usages

```
fun formatTimestampToDateString(timestamp: Long, locale: Locale = Locale.getDefault()): String {
    val localDate = Instant.ofEpochMilli( epochMilli = timestamp).atZone(ZoneId.of( zoneId = "UTC")).toLocalDate()
    val dateFormatter = DateTimeFormatter.ofPattern( pattern = "dd MMM yyyy", locale)
    return localDate.format(dateFormatter)
}
```

1 Usage

```
fun formatLocalDateToString(date: LocalDate, locale: Locale = Locale.getDefault()): String {
    val dateFormatter = DateTimeFormatter.ofPattern( pattern = "dd MMM yyyy", locale)
    return date.format(dateFormatter)
}
```

5 Usages

```
fun formatTotalMinutes(totalMinutes: Int): String {
    val hours = totalMinutes / 60
    val minutes = totalMinutes % 60
    return "${hours}h ${minutes}m"
}
```

- **2. Resiliencia de Red y Experiencia de Usuario (UX):** El hallazgo más significativo se produjo en las pruebas de latencia simulada (CP-21). Se evidenció que, ante una caída del servidor, la aplicación quedaba bloqueada indefinidamente en un estado de carga ("Saving..."). Para solucionar esto, se implementó una política de *Timeout* global de 10 segundos en el cliente HTTP (OkHttpClient), garantizando que las peticiones estancadas se aborren de forma segura y se notifique visualmente al usuario, protegiendo la fluidez de la navegación.

```

1 Usage
@Provides
@Singleton
fun provideOkHttpClient(authInterceptor: Interceptor): OkHttpClient {
    return OkHttpClient.Builder()
        .addInterceptor(authInterceptor)
        .connectTimeout( timeout = 10, unit = TimeUnit.SECONDS)
        .readTimeout( timeout = 10, unit = TimeUnit.SECONDS)
        .writeTimeout( timeout = 10, unit = TimeUnit.SECONDS)
        .build()
}

```

- **3. Blindaje de Seguridad y Ética de Datos:** Se detectaron vulnerabilidades iniciales en el ingreso de datos que podrían comprometer la base de datos. Como acción correctiva, se endurecieron las expresiones regulares (Regex) en los formularios, bloqueando correos electrónicos malformados (falsos positivos) y forzando la presencia obligatoria de caracteres alfanuméricos en las contraseñas.

```

6 Usages
fun getEmailError(email: String): String? {
    if (email.isBlank()) {
        return "Email cannot be empty"
    }

    val emailRegex = """"^[A-Za-z0-9+_.-]+@([A-Za-z0-9-]+\.)+[A-Za-z]{2,}$""".toRegex()

    if (!emailRegex.matches( input = email)) {
        return "Invalid email format"
    }
    return null
}

```

```

5 Usages
fun isPassValid(pass: String): Boolean {
    return pass.length >= 7 &&
        pass.any {it.isDigit()} &&
        pass.any {!it.isLetterOrDigit()} &&
        pass.any { it.isLetter()}
}

```

- 4. Código innecesario:** Durante el desarrollo del plan de pruebas se identificó una cantidad significativa de código innecesario en el backend que no es utilizado por el flujo real del sistema, no acata a ningún requisito funcional e incluso representa una potencial vulnerabilidad de seguridad. Concretamente, los controladores Country, Film, Genre y partes del manejo de Job y Person exponen endpoints REST de tipo CRUD (crear, eliminar, consultar) o presentan código sin uso sobre entidades que en la práctica se gestionan directamente desde la base de datos, ya que corresponden a datos maestros referencial. Esto significa que existen endpoints públicos a la API que permitirían a un usuario autenticado modificar el catálogo de géneros o eliminar países sin que el sistema haga uso real de esa funcionalidad. Como resultado de este hallazgo, se planea eliminar dichos controladores y servicios asociados en una próxima iteración, después de consultar sobre el tema con el profesor guía, reduciendo así la superficie de ataque del backend, simplificando el código y eliminando puntos de entrada que podrían ser explotados o utilizados accidentalmente para corromper datos críticos del sistema.

16. CONCLUSIÓN

El presente Plan de Pruebas Funcionales ha permitido establecer una estrategia estructurada y exhaustiva para asegurar la calidad, la estabilidad y una correcta experiencia de usuario (UX) en la aplicación Android KinoStats. A través de la definición y trazabilidad de los casos de prueba, se ha logrado cubrir de manera efectiva los Requisitos Funcionales y No Funcionales planteados para las historias de usuario principales del proyecto: Autenticación de cuentas (H1), Gestión y calificación de registros (H2), Visualización del dashboard de estadísticas (H3) y la administración de listas personalizadas (H4).

De manera complementaria, se diseñó e implementó una suite de pruebas dedicada al backend desarrollado en Spring Boot, compuesta por 41 casos de prueba que validan de forma aislada la lógica de negocio de los servicios mediante pruebas unitarias con Mockito, la correcta ejecución de las consultas JPQL personalizadas a través de pruebas de integración con `@DataJpaTest` sobre una base H2 en memoria, el comportamiento de los endpoints REST y la configuración de seguridad mediante pruebas con MockMvc y Spring Security Test, y la integración con servicios externos como TMDB utilizando MockWebServer para simular sus respuestas sin consumir cuota de la API real. Esta suite permite verificar que el backend cumple con su contrato funcional antes de que la aplicación cliente lo consuma, asegurando que cualquier defecto se detecte en la capa donde se origina.

Al abarcar tanto pruebas unitarias (sin despliegue) para aislar la lógica de negocio subyacente en la arquitectura MVVM, como pruebas instrumentadas (con despliegue) enfocadas en la interfaz construida con Jetpack Compose y sus flujos de navegación, este documento garantiza un enfoque de validación integral. Además, la estrategia de ejecución dividida en 4 ciclos progresivos asegura la detección temprana de fallos, la validación de la resiliencia del sistema ante errores de red y el cumplimiento estricto de las reglas de seguridad.

En definitiva, la aplicación rigurosa de este plan dota al equipo de KinoStats de los mecanismos y evidencias necesarias para certificar que el software construido no solo satisface los criterios de aceptación acordados con el cliente, sino que también proporciona una base técnica robusta, segura y tolerante a fallos para su despliegue final.